

Guia Passo a Passo: Interface e Comandos da Calculadora React Native

Este guia destrincha o funcionamento da calculadora, ilustrando como o React Native estrutura a tela e explicando cada comando do código linha por linha.

1. Criação do Projeto e Comandos Iniciais

Abaixo estão os comandos executados no terminal e a explicação técnica do que eles fazem no seu computador:

- **npx create-expo-app minha-calculadora:** O npx aciona o gerenciador de pacotes do Node.js. O create-expo-app baixa um modelo pronto, criando as pastas, configurações nativas (iOS/Android) e instalando as bibliotecas essenciais para o app rodar sem configurações manuais.
- **cd minha-calculadora:** O comando cd (Change Directory) altera o diretório atual do terminal, "entrando" na pasta do recém-criado projeto.
- **npx expo start:** Inicializa o Metro Bundler, que atua como um servidor de desenvolvimento. Ele processa o código JavaScript e o envia para o aplicativo Expo Go no seu smartphone ou para um emulador.

2. Componentes de Tela: O que você enxerga

No React Native, a interface não é feita com HTML. Usamos componentes nativos que se transformam em elementos visuais reais no iOS e no Android.

- **<View>:** É a base da interface. Funciona como um "caixote" transparente usado para agrupar e organizar os botões e textos na tela.
- **<Text>:** O único componente capaz de renderizar letras e números. Nenhum texto pode ficar solto na tela fora desta tag.
- **<TouchableOpacity>:** É o componente de interação (o nosso botão). Quando tocado, ele diminui a opacidade do que está dentro dele, fornecendo um feedback visual imediato de que o clique foi registrado.

3. O Layout da Tela (Design e Posição)

Abaixo apresentamos uma representação visual de como os elementos são estruturados. O teclado é dividido em "linhas" utilizando o modelo Flexbox.

DISPLAY DO RESULTADO (ex: 1234)			
C (Limpar)	+/-	%	÷
7	8	9	×
4	5	6	-
1	2	3	+
0		.	=

4. Lógica de Estado (Memória RAM da Calculadora)

A interface precisa de dados vivos para ser atualizada. Para isso, utilizamos o `useState` do React.

```
const [display, setDisplay] = useState('0');
const [previousValue, setPreviousValue] = useState(null);
const [operator, setOperator] = useState(null);
```

- **display:** É a variável que armazena a string que o usuário vê agora. Inicia com o valor '0'. `setDisplay` é o comando que altera esse valor e manda a tela se redesenhar automaticamente.
- **previousValue:** Quando o usuário clica em um operador (+, -, etc), nós armazenamos o que estava no display aqui, para podermos limpar o visor para o próximo número.

- **operator:** Armazena o tipo de operação matemática solicitada.

5. Dissecando os Comandos Interativos

Abaixo detalhamos as funções que executam a matemática da aplicação.

Adicionar um Número

```
const handleNumber = (number) => {  
  setDisplay(display === '0' ? number : display + number);  
};
```

Explicação: A função `handleNumber` recebe o número do botão que foi tocado. O operador condicional (ternário) avalia: O `display` atual é igual a `'0'`? Se sim, ele apenas substitui o `'0'` pelo número novo (ex: `'7'`). Se não, ele concatena o número existente com o novo (ex: `'7' + '5' = '75'`).

Clicar em uma Operação Matemática

```
const handleOperator = (op) => {  
  setOperator(op);  
  setPreviousValue(display);  
  setDisplay('0');  
};
```

Explicação: Esta função congela o estado atual para preparar a conta. Ela grava o símbolo clicado (`'+'`, `'-'`, etc.) no `operator`, move o valor atual da tela para `previousValue` e redefine a tela para `'0'` para receber a próxima digitação.

Processar o Resultado (Botão de Igual)

```
const handleEqual = () => {  
  const current = parseFloat(display);  
  const previous = parseFloat(previousValue);  
  if (!previousValue || !operator) return;  
  
  let result = 0;  
  // Operações lógicas  
  if (operator === '+') result = previous + current;  
  /* ... outras operações ... */  
  
  setDisplay(String(result));  
  setPreviousValue(null);  
  setOperator(null);  
};
```

Explicação dos Comandos:

1. **parseFloat():** Converte o texto da tela em um número real com casas decimais. Sem isso, o JavaScript iria concatenar textos ao invés de somá-los (ex: '2' + '2' viraria '22').
2. **if (!previousValue || !operator) return:** É uma barreira de segurança. Se o usuário clicar em "=" sem ter um número anterior ou um operador, a função simplesmente para (return), evitando um erro ou travamento.
3. **String(result):** Após calcular a matemática, precisamos transformar o número de volta em texto para o setDisplay, visto que o <Text> prefere lidar com strings nativamente.
4. **Limpendo variáveis:** Após calcular, o estado de previousValue e operator retorna a nulo, preparando a calculadora para uma conta completamente nova.

6. Estilização: Entendendo os Parâmetros

O React Native estiliza utilizando as propriedades da API StyleSheet, que usa a mesma lógica do CSS, mas escrita em CamelCase. Eis os comandos centrais explicados:

- **flexDirection: 'row':** A orientação padrão no celular é coluna (de cima para baixo). O comando 'row' força os botões de cada View a ficarem enfileirados lado a lado, horizontalmente.
- **flex: 1:** Ordena ao elemento que ocupe todo o espaço livre disponível. Se 4 botões usam flex: 1 na mesma linha, eles dividem o espaço de tela exatamente em 4 partes iguais (25% cada).
- **justifyContent: 'center':** Alinha o conteúdo verticalmente no centro do botão (dentro da estrutura Row/Flex).
- **alignItems: 'center':** Alinha o conteúdo horizontalmente no centro. A união deste comando com o justifyContent garante que o texto fique exatamente no meio da bolinha.
- **borderRadius: 50:** Arredonda as pontas de um elemento quadrado. Quando o valor é alto (como 50), o botão passa de quadrado a circular, criando o visual clássico do iOS.